

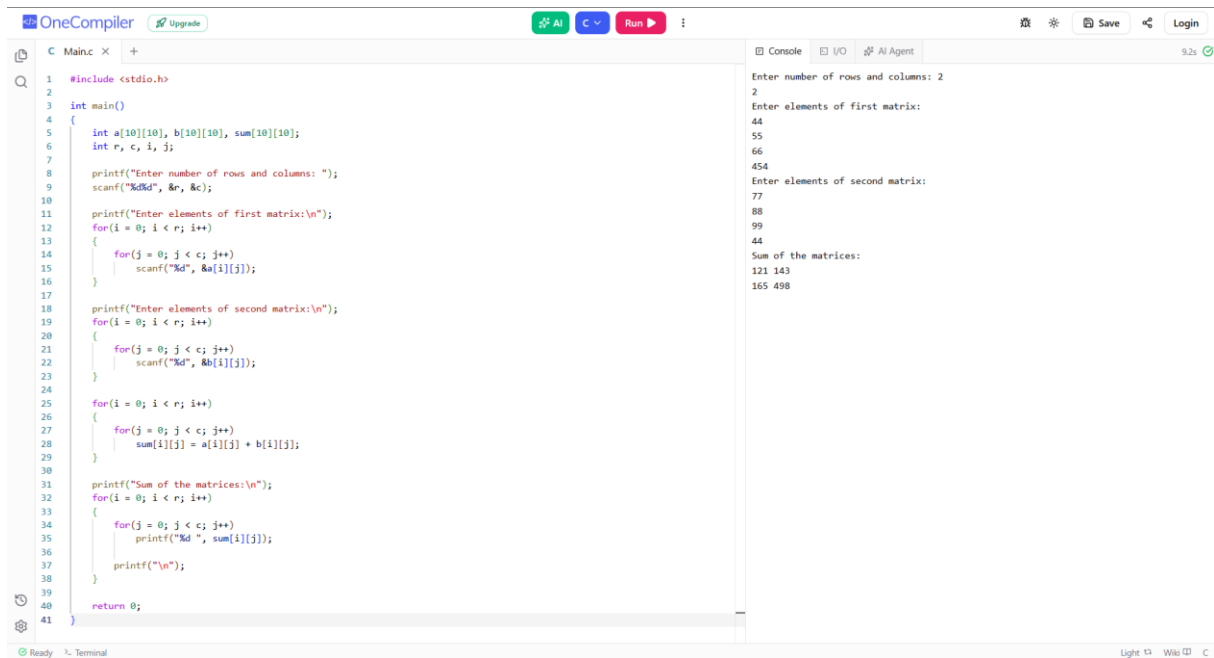
GITHUB ASSIGNMENT-1

IT23302 DATA STRUCTURES

NAME:- Farah Fathima K.M

ROLL NO:-2025115014

1. Sum of two matrices

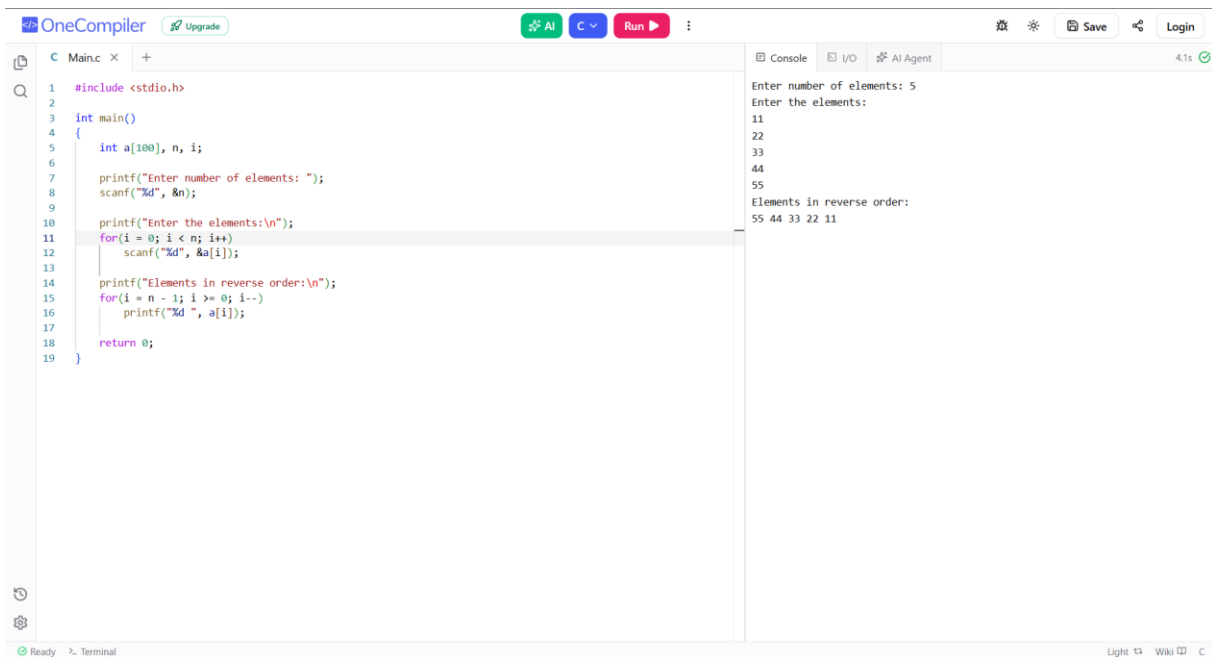


```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[10][10], b[10][10], sum[10][10];
6     int r, c, i, j;
7
8     printf("Enter number of rows and columns: ");
9     scanf("%d%d", &r, &c);
10
11     printf("Enter elements of first matrix:\n");
12     for(i = 0; i < r; i++)
13     {
14         for(j = 0; j < c; j++)
15             scanf("%d", &a[i][j]);
16     }
17
18     printf("Enter elements of second matrix:\n");
19     for(i = 0; i < r; i++)
20     {
21         for(j = 0; j < c; j++)
22             scanf("%d", &b[i][j]);
23     }
24
25     for(i = 0; i < r; i++)
26     {
27         for(j = 0; j < c; j++)
28             sum[i][j] = a[i][j] + b[i][j];
29     }
30
31     printf("Sum of the matrices:\n");
32     for(i = 0; i < r; i++)
33     {
34         for(j = 0; j < c; j++)
35             printf("%d ", sum[i][j]);
36         printf("\n");
37     }
38
39     return 0;
40 }
41
```

Console Output:

```
Enter number of rows and columns: 2
2
Enter elements of first matrix:
44
55
66
454
Enter elements of second matrix:
77
88
99
44
Sum of the matrices:
121 143
165 498
```

2. Write a C program to read n number of values in an array and display them in reverse order.



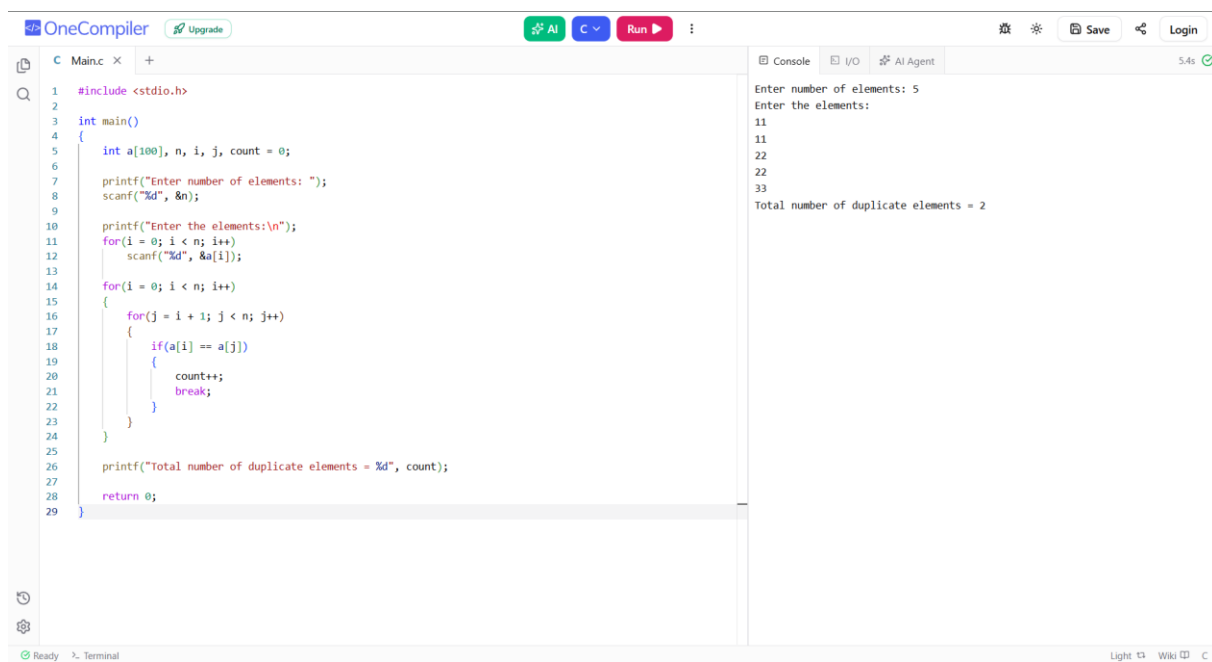
The screenshot shows the OneCompiler interface with a C program. The code reads a number of elements (5) and then reads those elements (11, 22, 33, 44, 55). It then prints them in reverse order (55, 44, 33, 22, 11).

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[100], n, i;
6
7     printf("Enter number of elements: ");
8     scanf("%d", &n);
9
10    printf("Enter the elements:\n");
11    for(i = 0; i < n; i++)
12        scanf("%d", &a[i]);
13
14    printf("Elements in reverse order:\n");
15    for(i = n - 1; i >= 0; i--)
16        printf("%d ", a[i]);
17
18    return 0;
19 }
```

Console output:

```
Enter number of elements: 5
Enter the elements:
11
22
33
44
55
Elements in reverse order:
55 44 33 22 11
```

3. Count the total number of duplicate elements in an array.



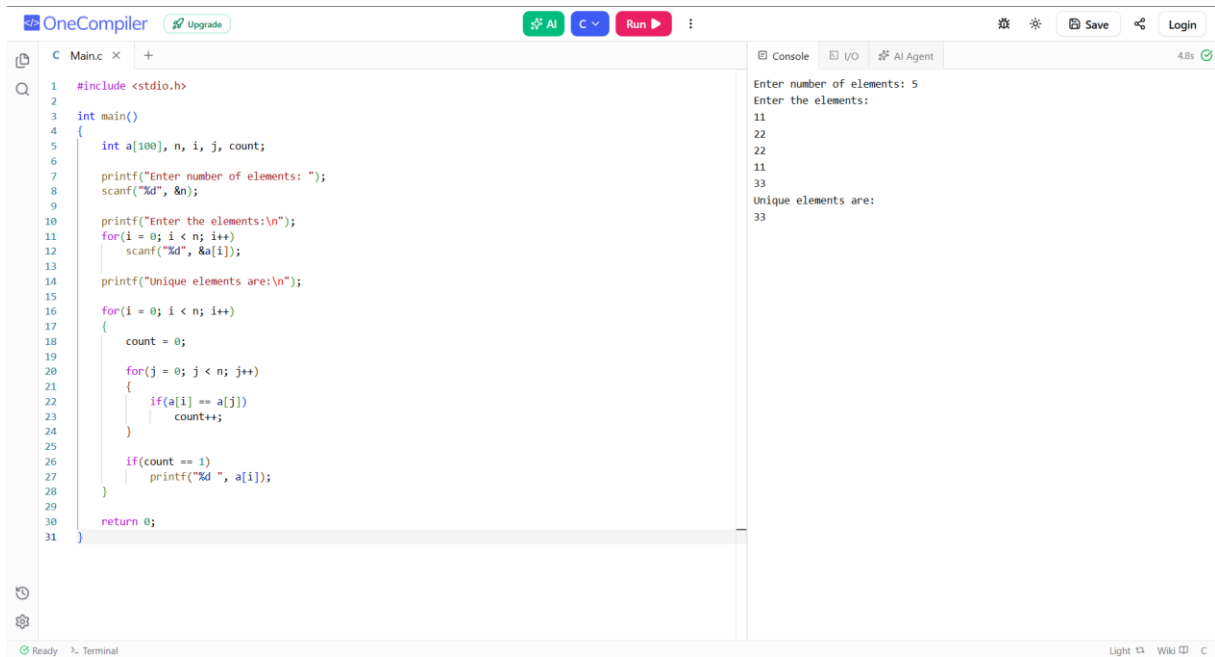
The screenshot shows the OneCompiler interface with a C program. The code reads a number of elements (5) and then reads those elements (11, 22, 33, 11, 22). It then counts the total number of duplicate elements (2) and prints the result.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[100], n, i, j, count = 0;
6
7     printf("Enter number of elements: ");
8     scanf("%d", &n);
9
10    printf("Enter the elements:\n");
11    for(i = 0; i < n; i++)
12        scanf("%d", &a[i]);
13
14    for(i = 0; i < n; i++)
15    {
16        for(j = i + 1; j < n; j++)
17        {
18            if(a[i] == a[j])
19            {
20                count++;
21                break;
22            }
23        }
24    }
25
26    printf("Total number of duplicate elements = %d", count);
27
28    return 0;
29 }
```

Console output:

```
Enter number of elements: 5
Enter the elements:
11
22
33
11
22
Total number of duplicate elements = 2
```

4. Print unique elements in an array:

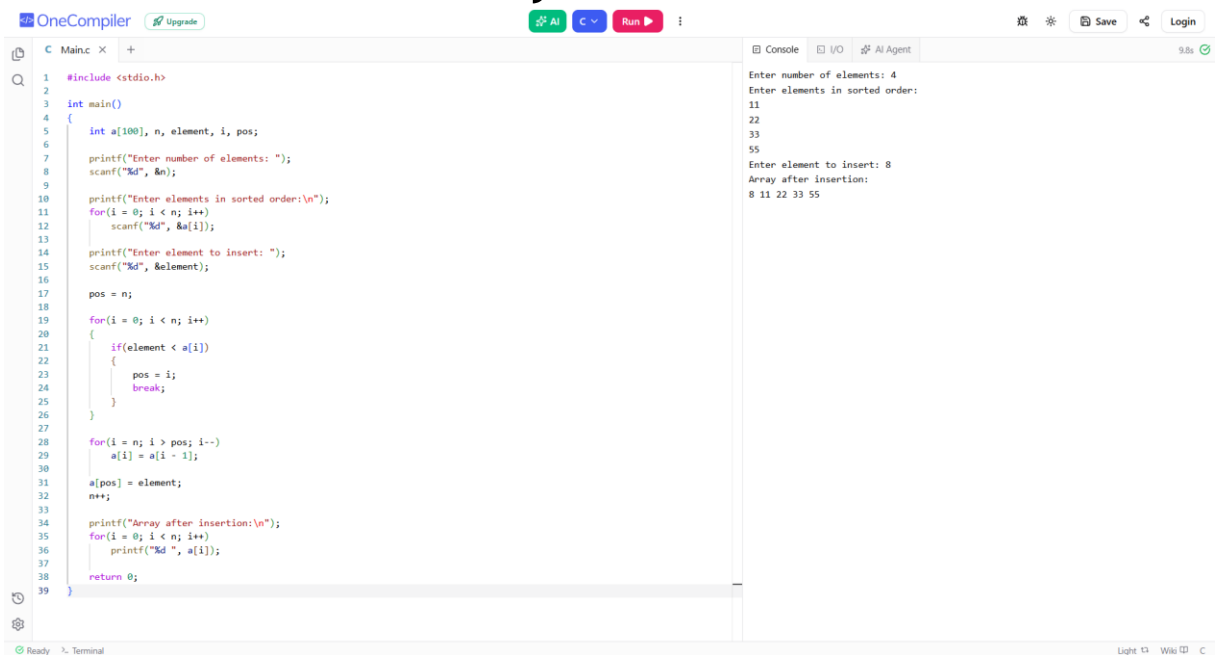


The screenshot shows the OneCompiler IDE with a C program to print unique elements from an array. The code uses nested loops to compare each element with all others and prints it only if it's unique.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[100], n, i, j, count;
6
7     printf("Enter number of elements: ");
8     scanf("%d", &n);
9
10    printf("Enter the elements:\n");
11    for(i = 0; i < n; i++)
12        scanf("%d", &a[i]);
13
14    printf("Unique elements are:\n");
15
16    for(i = 0; i < n; i++)
17    {
18        count = 0;
19
20        for(j = 0; j < n; j++)
21        {
22            if(a[i] == a[j])
23                count++;
24        }
25
26        if(count == 1)
27            printf("%d ", a[i]);
28    }
29
30    return 0;
31 }
```

The console output shows the user entering 5 elements: 11, 22, 22, 11, 33. The program then prints the unique elements: 11, 22, 33.

5. Insert in sorted array

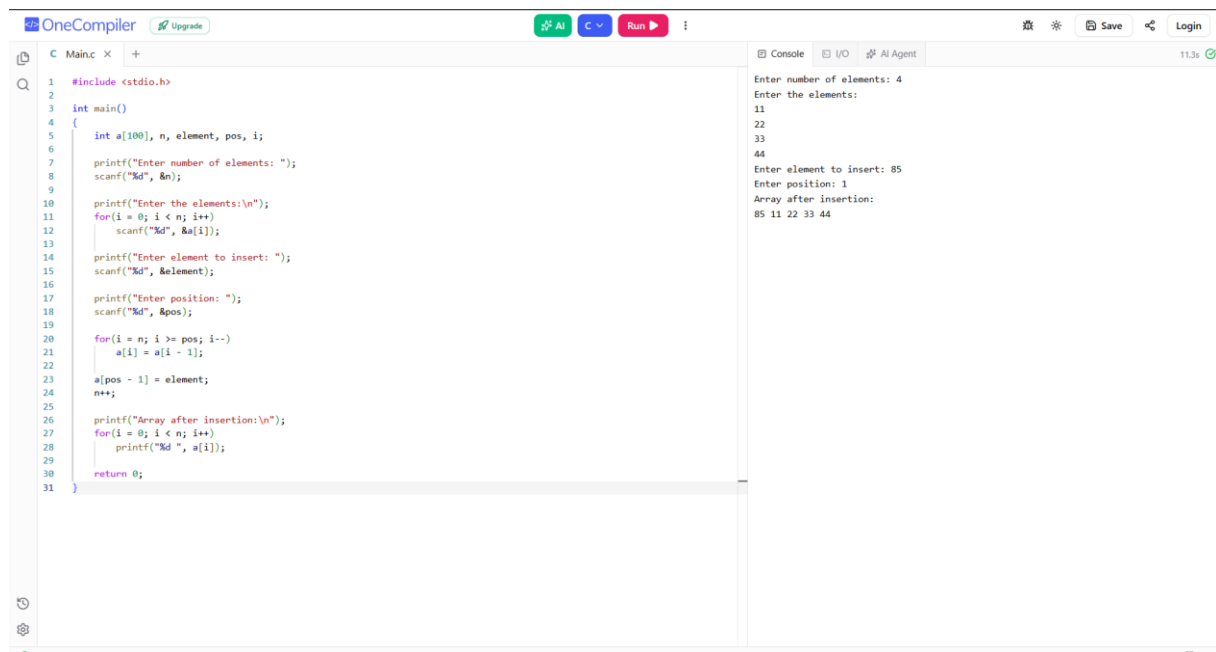


The screenshot shows the OneCompiler IDE with a C program to insert an element into a sorted array. The program finds the correct position for the new element and shifts the existing elements to the right.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[100], n, element, i, pos;
6
7     printf("Enter number of elements: ");
8     scanf("%d", &n);
9
10    printf("Enter elements in sorted order:\n");
11    for(i = 0; i < n; i++)
12        scanf("%d", &a[i]);
13
14    printf("Enter element to insert: ");
15    scanf("%d", &element);
16
17    pos = n;
18
19    for(i = 0; i < n; i++)
20    {
21        if(element < a[i])
22        {
23            pos = i;
24            break;
25        }
26    }
27
28    for(i = n; i > pos; i--)
29        a[i] = a[i - 1];
30
31    a[pos] = element;
32    n++;
33
34    printf("Array after insertion:\n");
35    for(i = 0; i < n; i++)
36        printf("%d ", a[i]);
37
38    return 0;
39 }
```

The console output shows the user entering 4 elements: 11, 22, 33, 55. Then, the user enters the element to insert: 8. The program then prints the array after insertion: 8 11 22 33 55.

6. Insert in sorted array.



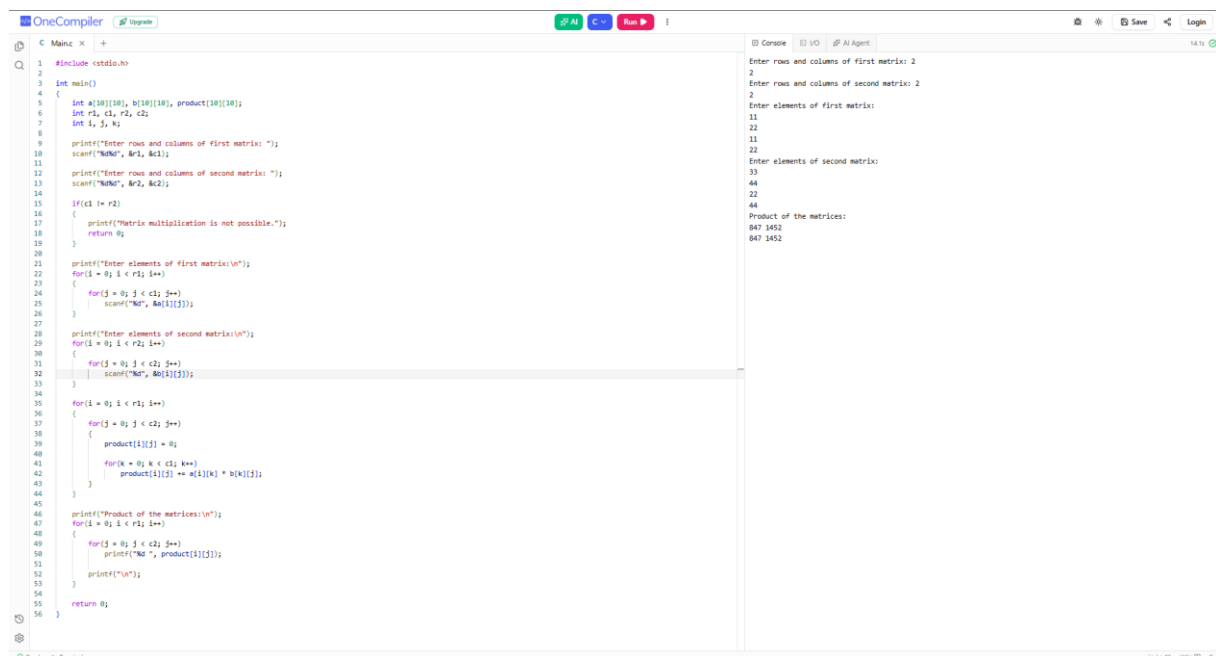
The screenshot shows the OneCompiler interface with a C program for inserting an element into a sorted array. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[100], n, element, pos, i;
6
7     printf("Enter number of elements: ");
8     scanf("%d", &n);
9
10    printf("Enter the elements:\n");
11    for(i = 0; i < n; i++)
12        scanf("%d", &a[i]);
13
14    printf("Enter element to insert: ");
15    scanf("%d", &element);
16
17    printf("Enter position: ");
18    scanf("%d", &pos);
19
20    for(i = n; i >= pos; i--)
21        a[i] = a[i - 1];
22
23    a[pos - 1] = element;
24    n++;
25
26    printf("Array after insertion:\n");
27    for(i = 0; i < n; i++)
28        printf("%d ", a[i]);
29
30    return 0;
31 }
```

The console output shows the following interaction:

```
Enter number of elements: 4
Enter the elements:
11
22
33
44
Enter element to insert: 85
Enter position: 1
Array after insertion:
85 11 22 33 44
```

7. Matrix multiplication.



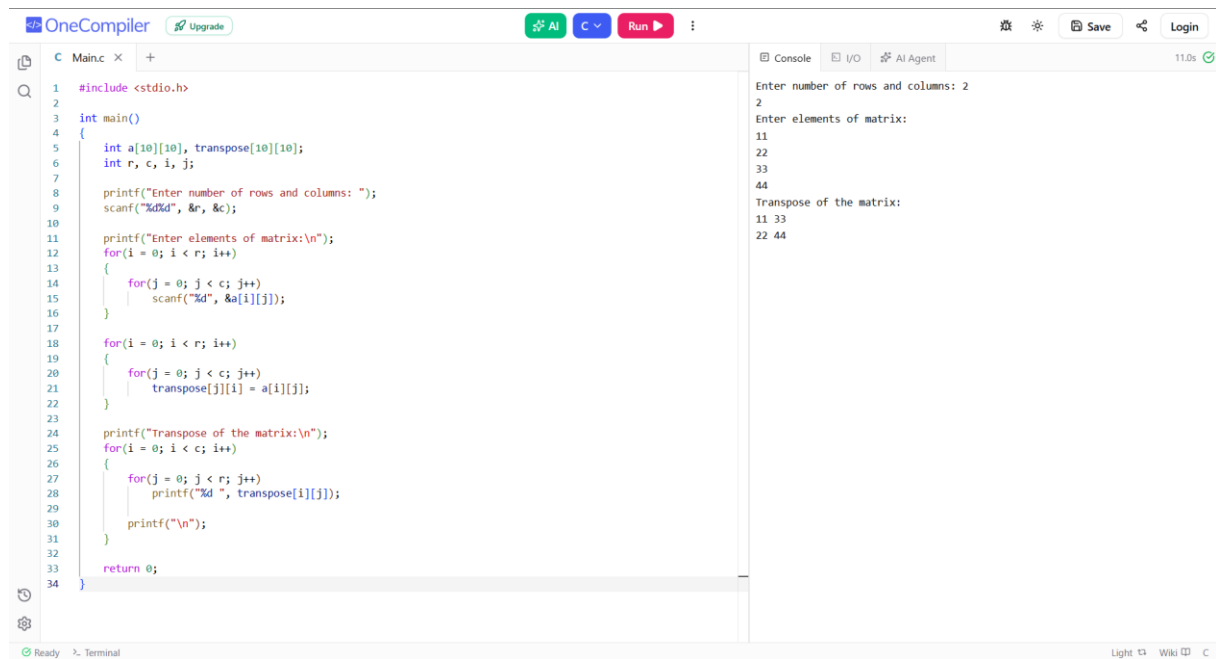
The screenshot shows the OneCompiler interface with a C program for matrix multiplication. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[10][10], b[10][10], product[10][10];
6     int r1, c1, r2, c2;
7     int i, j, k;
8
9     printf("Enter rows and columns of first matrix: ");
10    scanf("%d%d", &r1, &c1);
11
12    printf("Enter rows and columns of second matrix: ");
13    scanf("%d%d", &r2, &c2);
14
15    if(c1 != r2)
16    {
17        printf("Matrix multiplication is not possible.");
18        return 0;
19    }
20
21    printf("Enter elements of first matrix:\n");
22    for(i = 0; i < r1; i++)
23    {
24        for(j = 0; j < c1; j++)
25            scanf("%d", &a[i][j]);
26    }
27
28    printf("Enter elements of second matrix:\n");
29    for(i = 0; i < r2; i++)
30    {
31        for(j = 0; j < c2; j++)
32            scanf("%d", &b[i][j]);
33    }
34
35    for(i = 0; i < r1; i++)
36    {
37        for(j = 0; j < c2; j++)
38        {
39            product[i][j] = 0;
40            for(k = 0; k < c1; k++)
41                product[i][j] += a[i][k] * b[k][j];
42        }
43    }
44
45    printf("Product of the matrices:\n");
46    for(i = 0; i < r1; i++)
47    {
48        for(j = 0; j < c2; j++)
49            printf("%d ", product[i][j]);
50        printf("\n");
51    }
52
53    return 0;
54 }
55
56 }
```

The console output shows the following interaction:

```
Enter rows and columns of first matrix: 2
2
Enter rows and columns of second matrix: 2
2
Enter elements of first matrix:
11
22
33
44
Enter elements of second matrix:
33
44
22
44
Product of the matrices:
847 1452
847 1452
```

8. Matrix transpose.



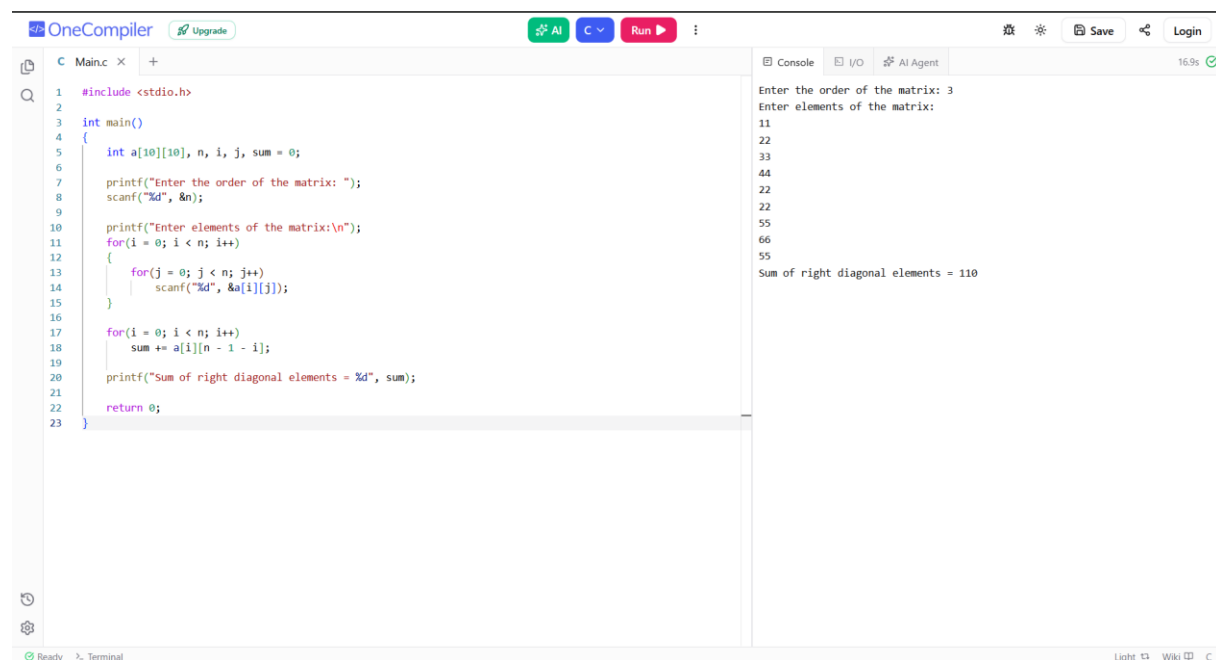
The screenshot shows the OneCompiler IDE with a C program for matrix transpose. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[10][10], transpose[10][10];
6     int r, c, i, j;
7
8     printf("Enter number of rows and columns: ");
9     scanf("%d%d", &r, &c);
10
11     printf("Enter elements of matrix:\n");
12     for(i = 0; i < r; i++)
13     {
14         for(j = 0; j < c; j++)
15             scanf("%d", &a[i][j]);
16     }
17
18     for(i = 0; i < r; i++)
19     {
20         for(j = 0; j < c; j++)
21             transpose[j][i] = a[i][j];
22     }
23
24     printf("Transpose of the matrix:\n");
25     for(i = 0; i < c; i++)
26     {
27         for(j = 0; j < r; j++)
28             printf("%d ", transpose[i][j]);
29         printf("\n");
30     }
31
32     return 0;
33 }
```

The console output shows the program execution:

```
Enter number of rows and columns: 2
2
Enter elements of matrix:
11
22
33
44
Transpose of the matrix:
11 33
22 44
```

9. Right diagonal sum



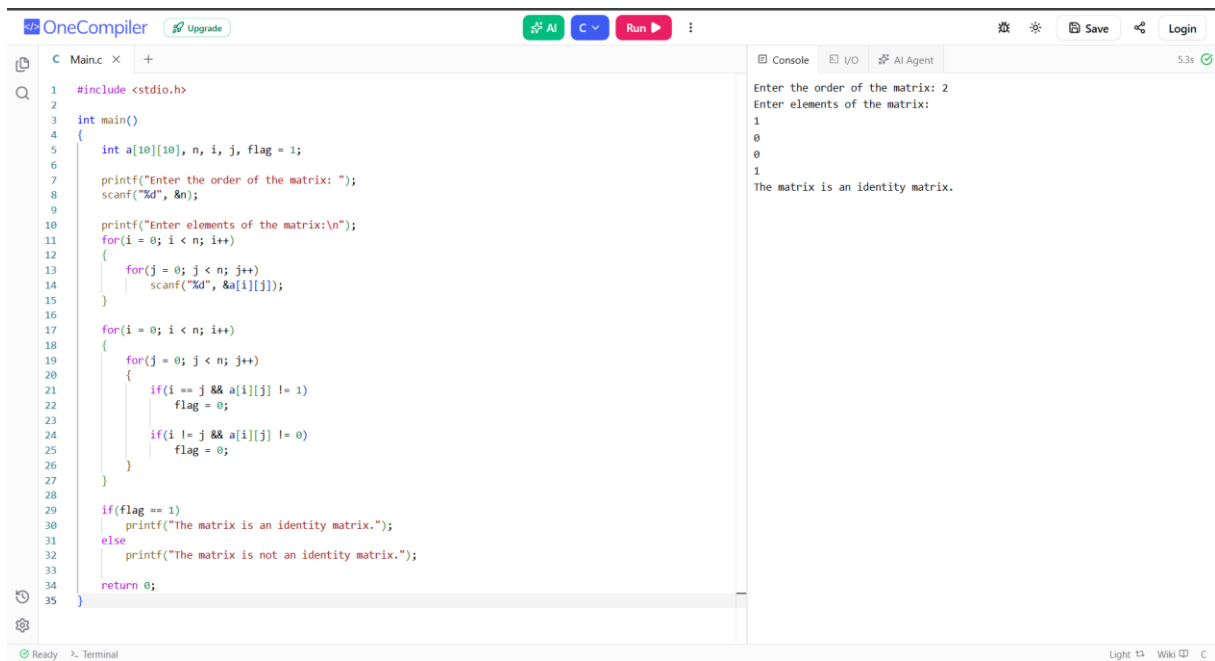
The screenshot shows the OneCompiler IDE with a C program for calculating the right diagonal sum of a matrix. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a[10][10], n, i, j, sum = 0;
6
7     printf("Enter the order of the matrix: ");
8     scanf("%d", &n);
9
10    printf("Enter elements of the matrix:\n");
11    for(i = 0; i < n; i++)
12    {
13        for(j = 0; j < n; j++)
14            scanf("%d", &a[i][j]);
15    }
16
17    for(i = 0; i < n; i++)
18        sum += a[i][n - 1 - i];
19
20    printf("Sum of right diagonal elements = %d", sum);
21
22    return 0;
23 }
```

The console output shows the program execution:

```
Enter the order of the matrix: 3
Enter elements of the matrix:
11
22
33
44
22
22
55
66
55
Sum of right diagonal elements = 110
```

10. Check for identity matrix



The screenshot shows the OneCompiler IDE with a C program to check if a matrix is an identity matrix. The code prompts the user to enter the order of the matrix (2) and then the elements of the matrix (1, 0, 0, 1). The program then checks if the matrix is an identity matrix and prints the result.

```
#include <stdio.h>

int main()
{
    int a[10][10], n, i, j, flag = 1;

    printf("Enter the order of the matrix: ");
    scanf("%d", &n);

    printf("Enter elements of the matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            if(i == j && a[i][j] != 1)
                flag = 0;

            if(i != j && a[i][j] != 0)
                flag = 0;
        }
    }

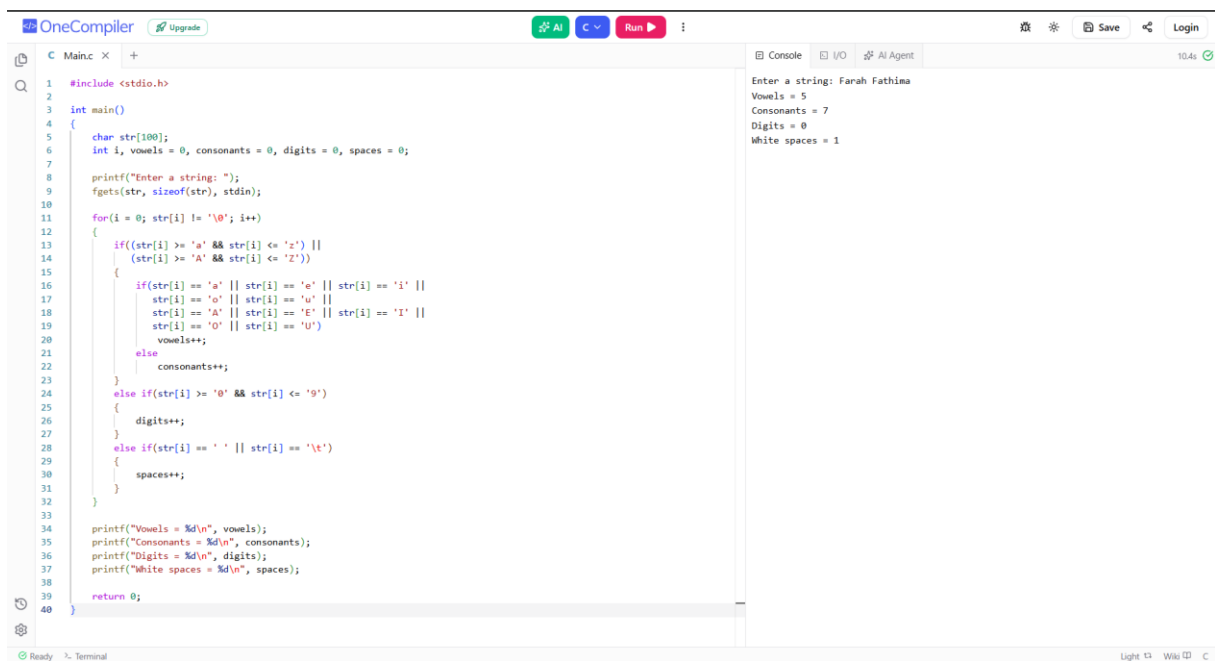
    if(flag == 1)
        printf("The matrix is an identity matrix.");
    else
        printf("The matrix is not an identity matrix.");

    return 0;
}
```

Console output:

```
Enter the order of the matrix: 2
Enter elements of the matrix:
1
0
0
1
The matrix is an identity matrix.
```

11. To find out number of vowels, consonants, digits, and white spaces



The screenshot shows the OneCompiler IDE with a C program to count the number of vowels, consonants, digits, and white spaces in a string. The code prompts the user to enter a string (Farah Fathima) and then counts the characters. The program then prints the counts for vowels (5), consonants (7), digits (0), and white spaces (1).

```
#include <stdio.h>

int main()
{
    char str[100];
    int i, vowels = 0, consonants = 0, digits = 0, spaces = 0;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    for(i = 0; str[i] != '\0'; i++)
    {
        if((str[i] >= 'a' && str[i] <= 'z') ||
           (str[i] >= 'A' && str[i] <= 'Z'))
        {
            if(str[i] == 'a' || str[i] == 'e' || str[i] == 'i' ||
               str[i] == 'o' || str[i] == 'u' ||
               str[i] == 'A' || str[i] == 'E' || str[i] == 'I' ||
               str[i] == 'O' || str[i] == 'U')
                vowels++;
            else
                consonants++;
        }
        else if(str[i] >= '0' && str[i] <= '9')
        {
            digits++;
        }
        else if(str[i] == ' ' || str[i] == '\t')
        {
            spaces++;
        }
    }

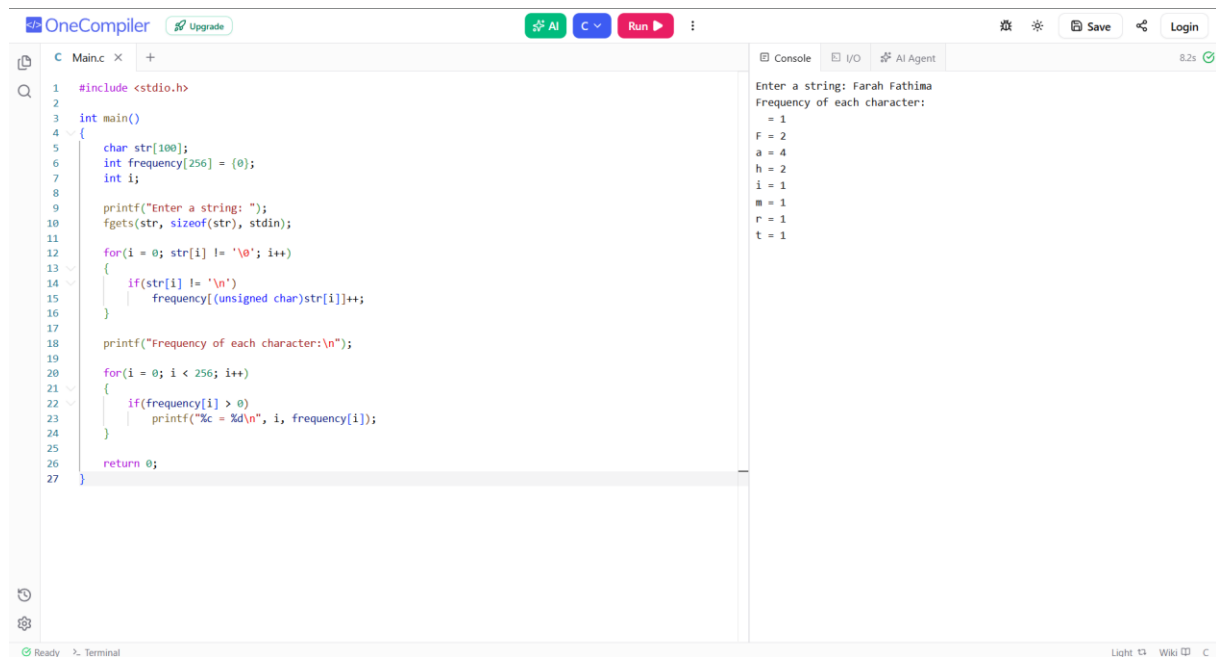
    printf("Vowels = %d\n", vowels);
    printf("Consonants = %d\n", consonants);
    printf("Digits = %d\n", digits);
    printf("White spaces = %d\n", spaces);

    return 0;
}
```

Console output:

```
Enter a string: Farah Fathima
Vowels = 5
Consonants = 7
Digits = 0
White spaces = 1
```

12. Frequency of character in a string.



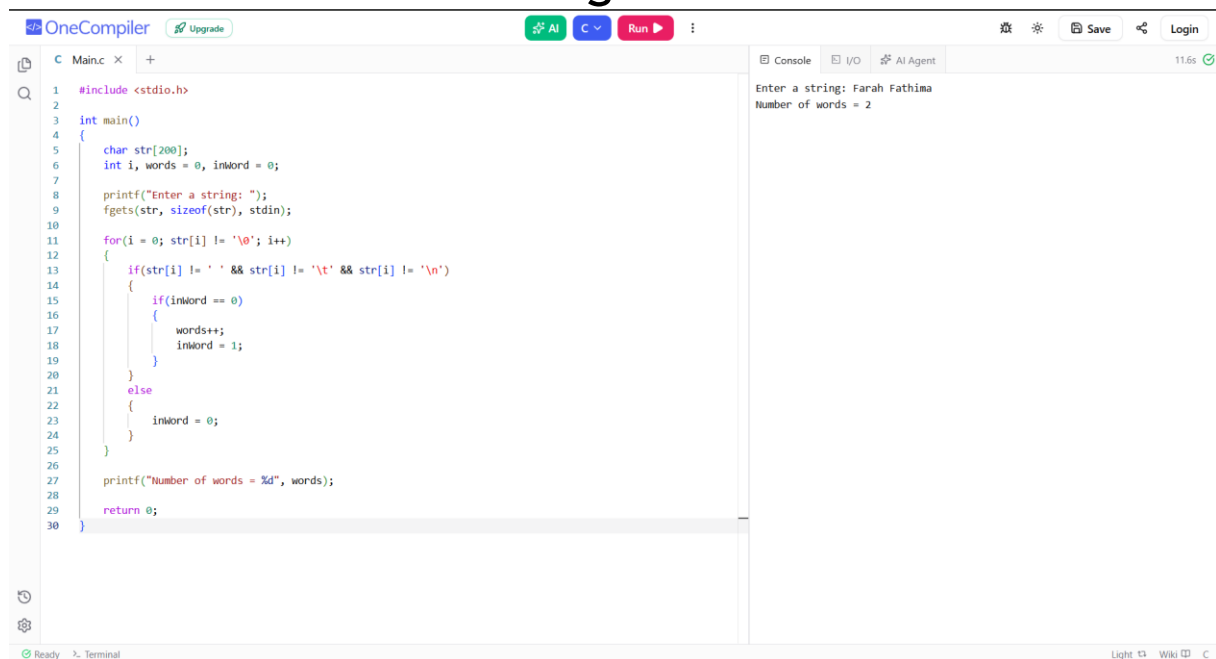
The screenshot shows the OneCompiler IDE with a C program to calculate the frequency of characters in a string. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char str[100];
6     int frequency[256] = {0};
7     int i;
8
9     printf("Enter a string: ");
10    fgets(str, sizeof(str), stdin);
11
12    for(i = 0; str[i] != '\0'; i++)
13    {
14        if(str[i] != '\n')
15            frequency[(unsigned char)str[i]]++;
16    }
17
18    printf("Frequency of each character:\n");
19
20    for(i = 0; i < 256; i++)
21    {
22        if(frequency[i] > 0)
23            printf("%c = %d\n", i, frequency[i]);
24    }
25
26    return 0;
27 }
```

The console output shows the input string "Farah Fathima" and the frequency of each character:

```
Enter a string: Farah Fathima
Frequency of each character:
 = 1
F = 2
a = 4
h = 2
i = 1
r = 1
t = 1
```

13. Count words in a string

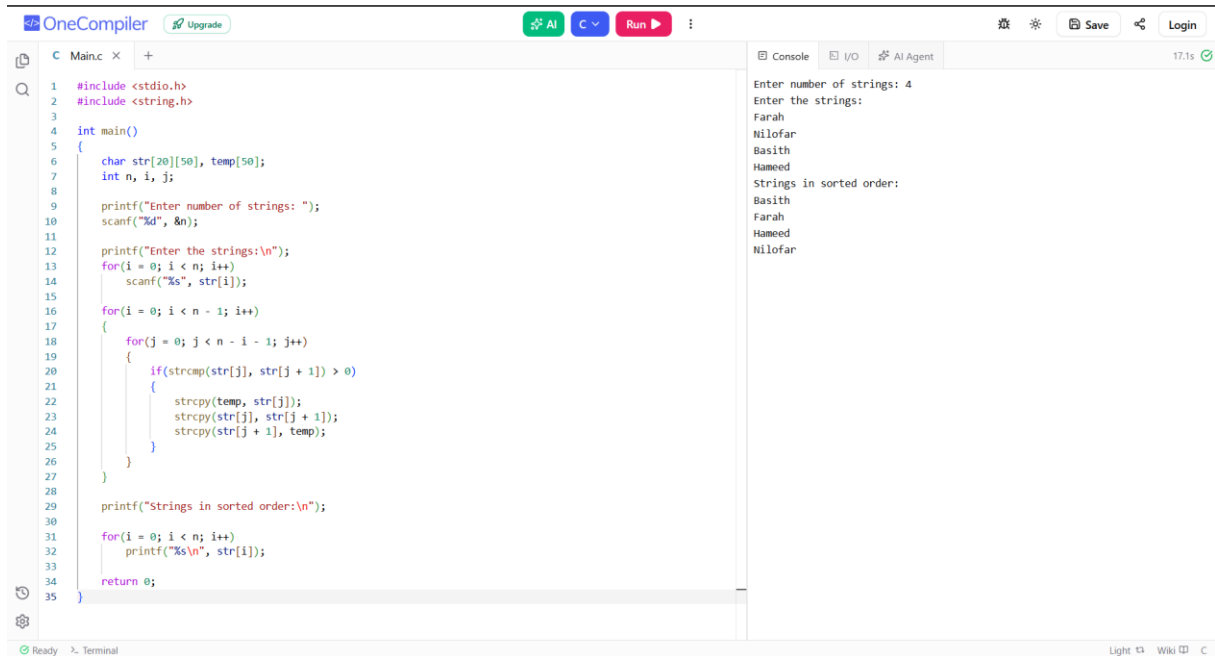


The screenshot shows the OneCompiler IDE with a C program to count the number of words in a string. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char str[200];
6     int i, words = 0, inWord = 0;
7
8     printf("Enter a string: ");
9     fgets(str, sizeof(str), stdin);
10
11    for(i = 0; str[i] != '\0'; i++)
12    {
13        if(str[i] != ' ' && str[i] != '\t' && str[i] != '\n')
14        {
15            if(inWord == 0)
16            {
17                words++;
18                inWord = 1;
19            }
20        }
21        else
22        {
23            inWord = 0;
24        }
25    }
26
27    printf("Number of words = %d", words);
28
29    return 0;
30 }
```

The console output shows the input string "Farah Fathima" and the result: "Number of words = 2".

14. Sort string array



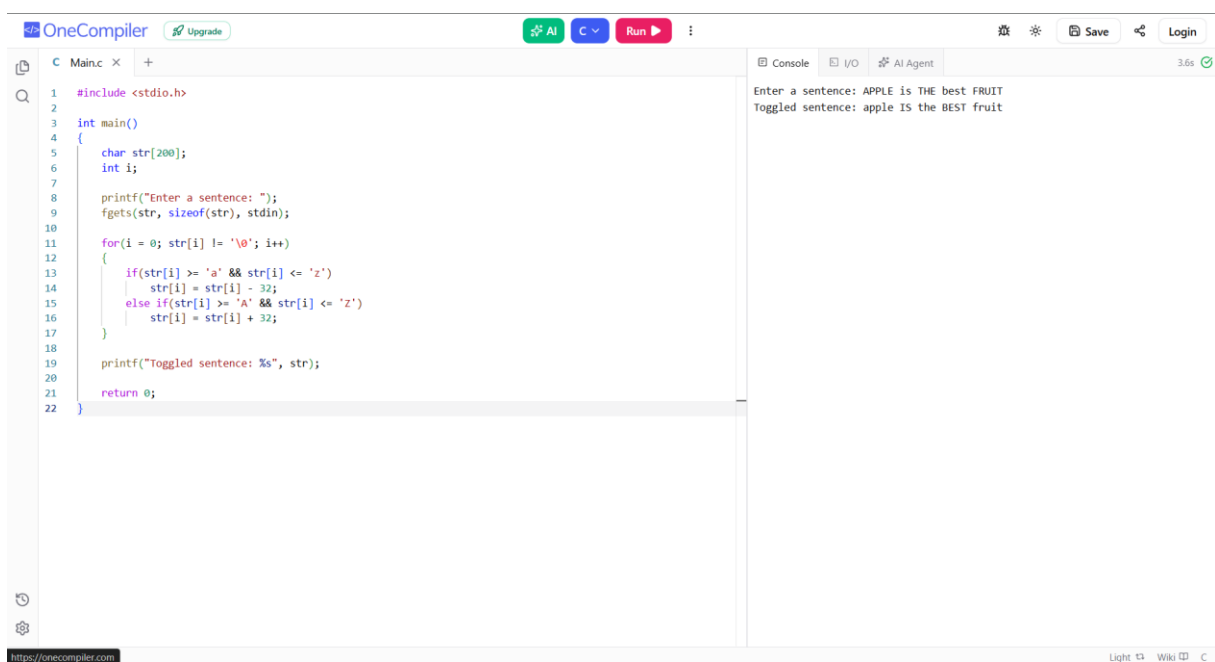
The screenshot shows the OneCompiler IDE with a C program that sorts an array of strings. The code is as follows:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char str[20][50], temp[50];
7     int n, i, j;
8
9     printf("Enter number of strings: ");
10    scanf("%d", &n);
11
12    printf("Enter the strings:\n");
13    for(i = 0; i < n; i++)
14        scanf("%s", str[i]);
15
16    for(i = 0; i < n - 1; i++)
17    {
18        for(j = 0; j < n - i - 1; j++)
19        {
20            if(strcmp(str[j], str[j + 1]) > 0)
21            {
22                strcpy(temp, str[j]);
23                strcpy(str[j], str[j + 1]);
24                strcpy(str[j + 1], temp);
25            }
26        }
27    }
28
29    printf("Strings in sorted order:\n");
30
31    for(i = 0; i < n; i++)
32        printf("%s\n", str[i]);
33
34    return 0;
35 }
```

The console output shows the program execution:

```
Enter number of strings: 4
Enter the strings:
Farah
Nilofar
Basith
Hameed
Strings in sorted order:
Basith
Farah
Hameed
Nilofar
```

15. Toggle case of a sentence.



The screenshot shows the OneCompiler IDE with a C program that toggles the case of a sentence. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     char str[200];
6     int i;
7
8     printf("Enter a sentence: ");
9     fgets(str, sizeof(str), stdin);
10
11    for(i = 0; str[i] != '\0'; i++)
12    {
13        if(str[i] >= 'a' && str[i] <= 'z')
14            str[i] = str[i] - 32;
15        else if(str[i] >= 'A' && str[i] <= 'Z')
16            str[i] = str[i] + 32;
17    }
18
19    printf("Toggled sentence: %s", str);
20
21    return 0;
22 }
```

The console output shows the program execution:

```
Enter a sentence: APPLE IS THE BEST FRUIT
Toggled sentence: apple IS THE BEST fruit
```